Optimization Basics for Machine Learning — Gradient Descent & Logistic Regression

Machine Learning — Lecture Notes (Session 3)

Formatted by: Heisenberg 😎

Index

- Recap: Linear Regression and the Closed-Form Solution — *where we relive least-squares glory before it abandons us.*
- Why We Need Iterative Optimization — *because not every function is polite enough to have a closed-form solution.*
- The Taylor Series Justification — *math's way of saying "trust me, small steps work."*
- The Gradient Descent Update Rule — *one line of algebra that trains half the internet's neural networks.*
- Gradient Descent in n Dimensions — *same idea, now with arrows instead of slopes.*
- The Gradient Descent Algorithm — *five lines of pseudocode, a thousand PhDs.*
- Worked Example: The Effect of Learning Rate — *watch one function get solved three different ways: slow, perfect, and chaotic.*
- Logistic Regression Recap — *the papaya problem: science's noblest fruit classification task.*
- The Sigmoid Function — *a very smooth way to squash numbers between 0 and 1.*
- Maximum Likelihood Estimation for Logistic Regression — *turning "predict probability" into "maximize probability."*
- Deriving the Gradient of the Log-Likelihood — *calculus, but it simplifies into something suspiciously elegant.*
- Gradient Ascent Algorithm for Logistic Regression — *descent's optimistic twin, climbing towards maximum probability.*
- The Scalability Problem — *why summing over a million data points is a terrible party trick.*
- Practice Questions — *five questions to make sure this didn't just wash over you.*

Recap: Linear Regression and the Closed-Form Solution

Before diving into iterative optimization, it's worth briefly revisiting the linear regression setup from the previous session, since gradient descent will later be applied to a problem where this convenient closed-form trick is no longer available.

Consider a training set:

$$S = \{(x_1, y_1), \dots, (x_n, y_n)\}$$

For the i-th data point, the predicted label is a linear combination of its features:

$$\hat{y}_i = a_1 x_{i1} + a_2 x_{i2} + \cdots + a_n x_{in}$$

Stacking all predictions and ground-truth labels into vectors:

$$\hat{y} = \begin{bmatrix} \hat{y}_1 \\ \vdots \\ \hat{y}_n \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}, \quad a = \begin{bmatrix} a_1 \\ \vdots \\ a_n \end{bmatrix}$$

The learning goal is to find the coefficient vector a that minimizes the squared distance between predictions and ground truth:

$$\min_{a_1,\dots,a_n} \|\hat{y} - y\|_2^2 = \min_a \|Aa - y\|_2^2$$

Here each prediction can be written compactly as $\hat{y}_i = a^\top x_i$, and the matrix A stacks every data point as a row:

$$A = \begin{bmatrix} x_1^\top \\ \vdots \\ x_n^\top \end{bmatrix}$$

So the objective function becomes:

$$f(a) = \|Aa - y\|_2^2$$

Taking the gradient with respect to a and setting it to zero:

$$\nabla f = 2A^\top Aa - 2A^\top y = 0$$

Solving this gives the **closed-form least-squares solution**:

$$a^* = (A^\top A)^{-1} A^\top y$$

This works beautifully for linear regression because the objective is **convex** with a **single global minimum**, which allows us to directly solve for the optimum algebraically. Logistic regression, as we will see later in this lecture, is not so cooperative — it has no closed-form solution, which is exactly why we need gradient descent.

Why We Need Iterative Optimization

In practice, many functions we want to minimize are far messier than the convex quadratic bowl of linear regression. They may have **multiple local minima** and highly complex expressions, making a closed-form solution to the global optimum impossible to derive.

So the natural question becomes: given a randomly initialized guess $x^{(0)}$ (which is generally *not* equal to the optimal solution x^*), how do we iteratively nudge this guess closer to the true optimum?

Picture a 1D function with a dip somewhere in the middle. At a starting point $x^{(0)}$, there are only two directions to move: left or right.

- Moving **right** (call this x_0^R) increases the function value.
- Moving **left** (call this x_0^L) decreases the function value.

This isn't a coincidence — it comes down to the **sign of the slope**:

- If the slope at the current point is **positive**, moving in the direction of increasing x increases $f(x)$.
- If the slope is **negative**, moving in the direction of increasing x decreases $f(x)$.

So, to minimize a function, you should always take a step in the direction opposite to the slope — that is, the **negative gradient direction**. This single observation is the seed of the entire gradient descent algorithm.

> **Note for Review:** This "always go against the slope" intuition is exactly why the algorithm is called *gradient **descent***. If you flip the sign and go *with* the slope, you get *gradient **ascent***, used for maximization problems (like the log-likelihood maximization we'll see in logistic regression later).

The Taylor Series Justification

Why does moving against the gradient actually guarantee a decrease in function value? We can prove this rigorously using a **first-order Taylor series expansion**.

For a small perturbation ϵ around a point x:

$$f(x+\epsilon) = f(x) + \frac{f'(x)}{1!}\epsilon + \frac{f''(x)}{2!}\epsilon^2 + \cdots$$

Since ϵ is chosen to be **very small**, all higher-order terms (quadratic, cubic, ...) become negligible, leaving the first-order approximation:

$$f(x+\epsilon) \approx f(x) + \epsilon f'(x)$$

We now want to find an ϵ such that:

$$f(x+\epsilon) < f(x)$$

Choose ϵ to point in the negative gradient direction, scaled by a **step size** $\eta > 0$ (also called the **learning rate**):

$$\epsilon = -\eta f'(x)$$

Substituting this back in:

$$f(x+\epsilon) = f(x) - \eta\,(f'(x))^2$$

Since $\eta > 0$ and $(f'(x))^2 \geq 0$, the term $\eta(f'(x))^2$ is always **non-negative**. Therefore:

$$f(x - \eta f'(x)) < f(x)$$

This confirms mathematically that stepping in the negative gradient direction is guaranteed to decrease the function value (for a sufficiently small step size).

The Gradient Descent Update Rule

From the Taylor series result, we get the fundamental **iterative update rule**:

$$x \longrightarrow x - \eta f'(x)$$

Unrolling this over multiple iterations, starting from an initial guess $x^{(0)}$:

$$x^{(1)} = x^{(0)} - \eta f'(x^{(0)})$$

$$x^{(2)} = x^{(1)} - \eta f'(x^{(1)})$$

$$x^{(3)} = x^{(2)} - \eta f'(x^{(2)})$$

In general, the recursive rule is:

$$x^{(t)} = x^{(t-1)} - \eta f'(x^{(t-1)})$$

Why doesn't this overshoot forever? As the solution gets closer to the local minimum, the slope $f'(x^{(t-1)})$ approaches zero (since at a minimum, the derivative is exactly 0). This means the update term $\eta f'(x^{(t-1)})$ shrinks towards zero as well, so consecutive solutions barely change — this naturally slows down and eventually halts the algorithm near the optimum.

This gives us a natural **stopping criterion**: stop iterating when the change between $x^{(t)}$ and $x^{(t-1)}$ becomes negligible, or when the gradient magnitude falls below a small threshold (e.g., 10^{-6}).

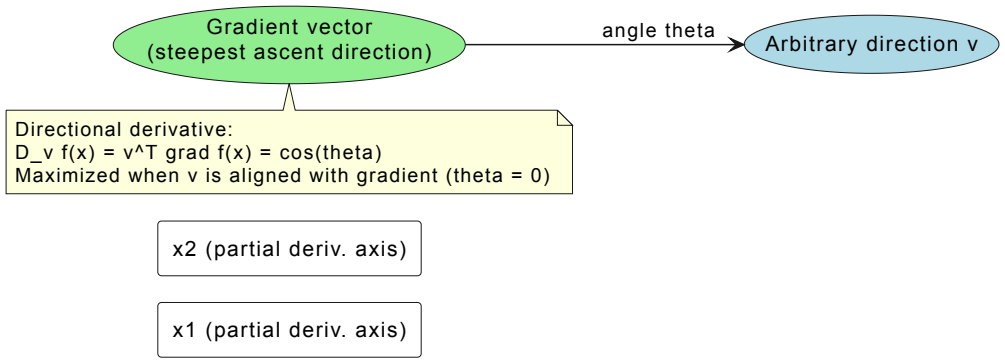
Gradient Descent in n Dimensions

Everything above was derived for a scalar function $f : \mathrm{R} \to \mathrm{R}$. Extending this to an n-dimensional function $f : \mathrm{R}^n \to \mathrm{R}$ requires only one conceptual change: replace the scalar derivative with the **gradient vector**.

$$\nabla f = [\frac{\partial f}{\partial x_1} \quad \frac{\partial f}{\partial x_2} \quad \cdots \quad \frac{\partial f}{\partial x_n}]^\top$$

The update rule becomes:

$$x^{(t)} = x^{(t-1)} - \eta \, \nabla f(x^{(t-1)})$$



Directional derivative: The rate of change of f in an arbitrary direction $\mathbf{v}$ at point $\mathbf{x}$ is defined as:

$$D_{\mathbf{v}} f(\mathbf{x}) = \mathbf{v}^\top \nabla f(\mathbf{x}) = \cos\theta$$

where θ is the angle between the direction $\mathbf{v}$ and the gradient $\nabla f(\mathbf{x})$ (assuming both are unit vectors).

This directional derivative is **maximized** when $\theta = 0°$, i.e., when $\mathbf{v}$ points in exactly the same direction as $\nabla f(\mathbf{x})$:

$$\max_{\mathbf{v}} \mathbf{v}^\top \nabla f(\mathbf{x}) = 1 \quad \text{when} \quad \mathbf{v} = \nabla f(\mathbf{x})$$

This is the formal justification for why the **gradient direction gives the steepest ascent**, and consequently, why the **negative gradient direction gives the steepest descent** — moving there decreases the function value the fastest among all possible directions, not just left/right as in the 1D case.

> **Note for Review:** In 1D there are only two directions to move (left or right), but in n dimensions there are infinitely many directions. The gradient tells you which single direction, out of infinitely many, decreases the function the fastest — this is the real power of generalizing from slopes to gradients.

The Gradient Descent Algorithm

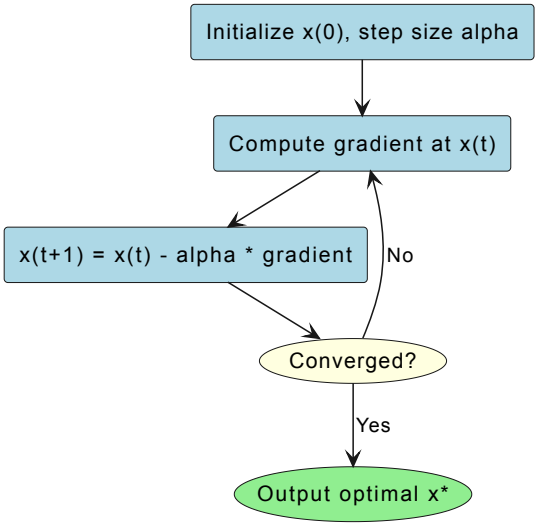
Putting it all together, the algorithm is remarkably short:

Algorithm: Gradient Descent

1. Initialize $\mathbf{x}^{(0)}$ and step size α.
2. $t \leftarrow 0$
3. **while** not converged **do**
4. $\quad \mathbf{x}^{(t+1)} \leftarrow \mathbf{x}^{(t)} - \alpha \nabla f(\mathbf{x}^t)$
5. $\quad t \leftarrow t + 1$
6. **end while**

Here α (also written η) is the **learning rate**, and convergence is typically checked either via the magnitude of the gradient or the difference between consecutive solutions.

Geometrically, if you view the contour plot of a bowl-shaped function from above, gradient descent traces a path of steps, each perpendicular to the local contour line, spiraling inward toward the global minimum.



This algorithm (or one of its many variants) underlies the training of virtually **all modern neural networks**.

Worked Example: The Effect of Learning Rate

To build intuition, consider the simple one-dimensional function:

$$f(x) = x^2 - 2x, \qquad f'(x) = 2x - 2$$

Solving $f'(x) = 0$ classically gives $x = 1$ as the optimal (global minimum) solution. We now verify that gradient descent, starting from an arbitrary initial guess, actually converges to this point — and that the **choice of learning rate α dramatically changes how (and whether) it converges.**

Case 1: Small learning rate ($\alpha = 0.1$)

Starting at $x^{(0)} = 3$:

- $f(x^{(0)}) = 3$, and $f'(x^{(0)}) = 2(3) - 2 = 4$

Iteration 1: $x^{(1)} = x^{(0)} - \alpha f'(x^{(0)}) = 3 - 0.1 \times 4 = 3 - 0.4 = 2.6$

Iteration 2: ($f'(2.6) = 2 \times 2.6 - 2 = 3.2$) $x^{(2)} = 2.6 - 0.1 \times 3.2 = 2.6 - 0.32 = 2.28$

Iteration 3: ($f'(2.28) = 2 \times 2.28 - 2 = 2.56$) $ x^{(3)} = 2.28 - 0.1 \times 2.56 = 2.28 - 0.256 = 2.024

$

After 3 iterations we've only gotten roughly halfway to $x^* = 1$. Notice the gradient magnitude is shrinking at each step ($4 \rightarrow 3.2 \rightarrow 2.56$), which is exactly why the step sizes shrink and convergence slows near the optimum. It would take several more iterations (roughly 7–8 more) to get very close to $x^* = 1$.

Case 2: Well-tuned learning rate ($\alpha = 0.5$)

Starting again at $x^{(0)} = 3$:

Iteration 1: $x^{(1)} = 3 - 0.5 \times 4 = 3 - 2 = 1$

Iteration 2: ($f'(1) = 2(1) - 2 = 0$) $x^{(2)} = 1 - 0.5 \times 0 = 1$

Here, gradient descent reaches the exact optimal solution $x^* = 1$ in a **single iteration**, and then stays there forever since the gradient at the optimum is zero.

Case 3: Learning rate too high ($\alpha = 1$)

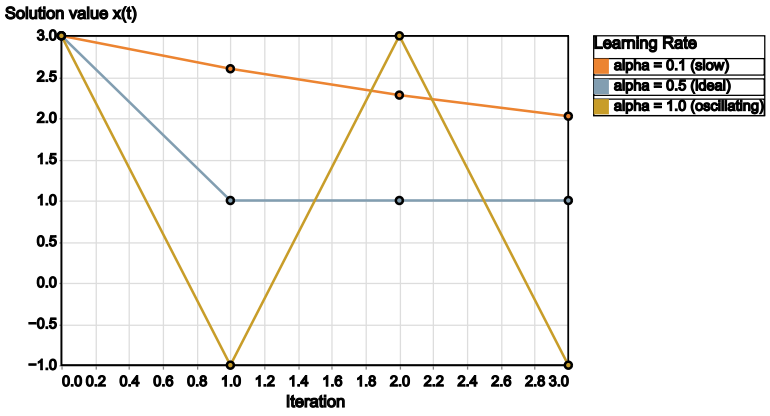
Starting at $x^{(0)} = 3$:

Iteration 1: $x^{(1)} = 3 - 1 \times 4 = -1$

The algorithm has now **overshot** the optimum entirely, landing on the opposite side of the curve.

Iteration 2: $(f'(-1) = 2(-1) - 2 = -4)$ $x^{(2)} = -1 - 1 \times (-4) = -1 + 4 = 3$

We've jumped straight back to the starting point! If you continue this process, the algorithm will keep **oscillating** between $x = 3$ and $x = -1$ forever, never settling at the true minimum $x^* = 1$.



Takeaways on Learning Rate

Learning Rate	Behavior	Outcome
Too small ($\alpha = 0.1$)	Converges, but very slowly	Needs many iterations
Well-tuned ($\alpha = 0.5$)	Converges quickly (here, in 1 step)	Ideal
Too high ($\alpha = 1$)	Oscillates between two points	Never converges
Even higher ($\alpha = 2$)	Diverges — function value increases without bound	Completely fails

> **Pro-Tip:** This is one of the most commonly tested exam concepts. Remember: a *too-small* learning rate wastes computation (slow convergence), while a *too-large* one can cause oscillation or outright divergence. For real neural networks with millions of parameters, tuning the learning rate is one of the most critical and difficult parts of training — this single-variable example already shows how delicate the balance is.

Logistic Regression Recap

Having built intuition for gradient descent on a simple quadratic, we now apply it to a problem that **cannot** be solved with a closed-form solution: **logistic regression**.

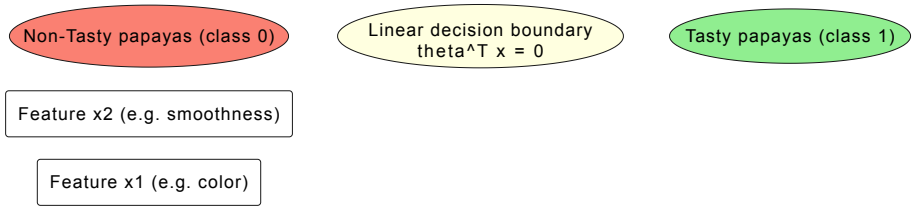
Consider the problem of classifying papayas into two classes: **"Tasty"** and **"Non-Tasty"**, based on features such as color and smoothness. Given a training dataset:

$$\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_m, y_m)\}$$

where $\mathbf{x}_i$ represents the i-th papaya's features and y_i its taste label, we use **binary labels**:

- $y_i = 0$ for a non-tasty papaya
- $y_i = 1$ for a tasty papaya

We assume the two classes are **linearly separable** — meaning a straight line (or hyperplane, in higher dimensions) can perfectly divide tasty from non-tasty papayas based on their features.



The goal is to find a **linear classifier**:

$$\theta^\top \mathbf{x} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_d x_d$$

The Sigmoid Function

Since we want our classifier's output to represent a probability (a value strictly between 0 and 1), a raw linear function $\theta^\top \mathbf{x}$ won't do — it's unbounded. Instead, we pass the linear score through the **sigmoid function**:

$$h_\theta(\mathbf{x}) = g(\theta^\top \mathbf{x}) = \frac{1}{1 + e^{-\theta^\top \mathbf{x}}}$$

As $z \to -\infty$, $g(z) \to 0$; as $z \to +\infty$, $g(z) \to 1$. This gives us exactly the [0, 1]-bounded output we need — if $h_\theta(\mathbf{x}) < 0.5$ we predict non-tasty, and if $h_\theta(\mathbf{x}) \geq 0.5$ we predict tasty.

Deriving the sigmoid's gradient (needed later for gradient descent):

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = \frac{dg(z)}{dz} = \frac{1}{(1 + e^{-z})^2} e^{-z} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \frac{1}{1 + e^{-z}} \left(1 - \frac{1}{1 + e^{-z}}\right)$$

$$g'(z) = g(z)(1 - g(z))$$

This elegant closed-form result — the derivative of the sigmoid is simply the sigmoid times one minus itself — is what makes logistic regression's gradient computations tractable.

> **Note for Review:** This "$g(z)(1 - g(z))$" trick is one of the most reused identities in deep learning — it appears in backpropagation for any network using a sigmoid activation. Memorize the derivation, not just the result.

Maximum Likelihood Estimation for Logistic Regression

We treat $h_\theta(\mathbf{x}_i)$ directly as a probability:

$$\mathrm{P}(y_i = 1 \mid \mathbf{x}_i) = h_\theta(\mathbf{x}_i), \qquad \mathrm{P}(y_i = 0 \mid \mathbf{x}_i) = 1 - h_\theta(\mathbf{x}_i)$$

These two cases can be elegantly combined into a single expression (a standard trick: whichever term doesn't apply gets raised to the power 0, vanishing):

$$\mathrm{P}(y_i \mid \mathbf{x}_i) = (h_\theta(\mathbf{x}_i))^{y_i} (1 - h_\theta(\mathbf{x}_i))^{1 - y_i}$$

Assuming the data points are **independent and identically distributed**, the joint likelihood over the whole dataset is the product of individual probabilities:

$$L(\theta) = p(y_1, \ldots, y_m \mid \mathbf{x}_1, \ldots, \mathbf{x}_m) = \prod_{i=1}^{m} p(y_i \mid \mathbf{x}_i) = \prod_{i=1}^{m} (h_\theta(\mathbf{x}_i))^{y_i} (1 - h_\theta(\mathbf{x}_i))^{1 - y_i}$$

Rather than maximizing this product directly (which is numerically awkward), we take the **log-likelihood**. Since the logarithm is a monotonically increasing function, it doesn't shift the location of the maximum — it only reshapes the function, which conveniently turns the product into a sum:

$$\ell(\theta) = \log(L(\theta)) = \sum_{i=1}^{m} y_i \log\left(h_\theta(\mathbf{x}_i)\right) + (1 - y_i) \log\left(1 - h_\theta(\mathbf{x}_i)\right)$$

This expression is exactly the (negative of the) **binary cross-entropy loss** commonly seen in classification tasks. Our goal is now to **maximize** $\ell(\theta)$ with respect to θ.

Since $\ell(\theta)$ involves the sigmoid function nested inside logarithms, it is a **highly non-linear** function of θ — unlike linear regression's quadratic objective, there is **no closed-form solution**. This is precisely why we need gradient-based optimization here.

Deriving the Gradient of the Log-Likelihood

To apply gradient ascent, we need $\frac{\partial \ell(\theta)}{\partial \theta_j}$ for each parameter θ_j:

$$\frac{\partial \ell(\theta)}{\partial \theta_j} = \sum_{i=1}^{m} y_i \frac{1}{h_\theta(\mathbf{x}_i)} \frac{\partial h_\theta(\mathbf{x}_i)}{\partial \theta_j} - (1 - y_i) \frac{1}{1 - h_\theta(\mathbf{x}_i)} \frac{\partial h_\theta(\mathbf{x}_i)}{\partial \theta_j}$$

$$= \sum_{i=1}^{m} \left(y_i \frac{1}{h_\theta(\mathbf{x}_i)} - (1 - y_i) \frac{1}{1 - h_\theta(\mathbf{x}_i)} \right) \frac{\partial h_\theta(\mathbf{x}_i)}{\partial \theta_j}$$

Using the sigmoid derivative identity $g'(z) = g(z)(1 - g(z))$ derived earlier:

$$= \sum_{i=1}^{m} \frac{(y_i - h_\theta(\mathbf{x}_i))}{h_\theta(\mathbf{x}_i)(1 - h_\theta(\mathbf{x}_i))} g(\theta^\top \mathbf{x}_i)(1 - g(\theta^\top \mathbf{x}_i)) \frac{\partial \theta^\top \mathbf{x}_i}{\partial \theta_j}$$

The $h_\theta(\mathbf{x}_i)(1 - h_\theta(\mathbf{x}_i))$ terms cancel beautifully with the sigmoid derivative, leaving a remarkably simple final result:

$$\frac{\partial \ell(\theta)}{\partial \theta_j} = \sum_{i=1}^{m} (y_i - h_\theta(\mathbf{x}_i)) x_{ij}$$

This clean form — "(actual minus predicted) times feature value, summed over all data points" — is what makes logistic regression's gradient so practical to compute despite the non-linearity of the original loss function.

Gradient Ascent Algorithm for Logistic Regression

Since we are **maximizing** the log-likelihood (not minimizing), the update uses a **plus** sign instead of the minus sign we saw in standard gradient descent — this variant is technically called **gradient ascent**:

Algorithm: Gradient Descent (Ascent) for Logistic Regression

1. **Input:** Training dataset $\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_m, y_m)\}$
2. Initialize $\theta^{(0)}$ and step size (learning rate) α.
3. $t \leftarrow 0$
4. **while** not converged **do**
5. **for** $j \in \{0, 1, 2, \ldots, d\}$ **do**
6. $\theta_j^{(t+1)} \leftarrow \theta_j^{(t)} + \alpha \sum_{i=1}^{m} (y_i - h_\theta(\mathbf{x}_i)) x_{ij}$
7. **end for**
8. $t \leftarrow t + 1$
9. **end while**
10. **Output:** Optimal classifier h_{θ^*}

> **Note for Review:** You can just as easily frame this as a *minimization* problem by minimizing the *negative* log-likelihood — flip the sign, and the update rule becomes a subtraction, matching the standard gradient descent form: $\theta_j^{(t+1)} \leftarrow \theta_j^{(t)} - \alpha \sum_i (h_\theta(\mathbf{x}_i) - y_i) x_{ij}$. Both formulations are equivalent and both appear frequently in textbooks — don't let the sign flip confuse you in an exam.

At convergence, the resulting θ^* defines the optimal logistic regression classifier — unlike linear regression, this required an **iterative** procedure since no closed-form solution exists.

The Scalability Problem

Looking closely at the gradient update rule:

$$\theta_j^{(t+1)} \leftarrow \theta_j^{(t)} + \alpha \sum_{i=1}^{m} (y_i - h_\theta(\mathbf{x}_i))x_{ij}$$

notice the summation over **all m training points**, computed **for every parameter θ_j**, on **every single iteration**.

This is fine for small datasets, but consider a modern large language model with training sets containing millions of data points and millions (or billions) of parameters, trained over millions of iterations. Performing a full summation over the entire dataset at every step becomes **computationally infeasible**.

This is a fundamental drawback of standard (a.k.a. "batch") gradient descent, and it directly motivates the need for more scalable variants of the algorithm — such as **Stochastic Gradient Descent (SGD)** and **mini-batch gradient descent** — which will be covered in the next module.

Practice Questions

1. Suppose you run gradient descent on $f(x) = x^2 - 2x$ starting from $x^{(0)} = 3$ but this time with $\alpha = 0.9$. Without fully computing every iteration, reason about whether the sequence will converge smoothly, oscillate, or diverge — and justify your answer using the relationship between α and the curvature of f.

2. The Taylor series argument for gradient descent relies on ignoring the second-order term $\frac{f''(x)}{2!}\epsilon^2$. For a function with very large curvature (large $f''(x)$) near the current point, explain why a *fixed* learning rate that works well for a "flatter" function might cause divergence here — and what this implies about the connection between curvature and the maximum "safe" learning rate.

3. In the logistic regression derivation, the final gradient expression $\sum_i (y_i - h_\theta(\mathbf{x}_i))x_{ij}$ looks almost identical in structure to the gradient of the linear regression squared-error loss. Given that logistic regression uses a highly non-linear sigmoid while linear regression is purely linear, why do you think their gradients end up looking so structurally similar? What role did the specific choice of the sigmoid function's derivative play in producing this simplification?

4. If you initialize logistic regression's gradient ascent with $\theta^{(0)} = \mathbf{0}$, what is $h_\theta(\mathbf{x}_i)$ for every data point at the very first iteration? Using this, reason about what the very first gradient update will look like, and whether the direction of that first update makes intuitive sense given the labels y_i.

5. The lecture mentions that summing over all m training points for every parameter, on every iteration, becomes computationally infeasible for datasets with millions of examples. Propose (in your own reasoning, without looking ahead) at least one modification to the update rule that could reduce this per-iteration computational cost, and discuss what trade-off your proposed modification might introduce in terms of convergence stability.